# SMART HEALTHCARE PATIENT APPOINTMENT AND MEDICAL RECORD MANAGEMENT SYSTEM USING C

C PROGRAMMING – INTERNAL ASSESSMENT ASSIGNMENT

TEAM 7

Team Members: Vikash J • Jafar Husen • Tharun Kumar

Faculty Guide: Irin Sherly

September 2026

## 1. PROBLEM STATEMENT

Design and implement a menu-driven C application for managing patient appointments and medical records in a multi-specialty hospital. The system should assist hospital administrators in maintaining patient details, doctor information, appointment schedules, medical history and priority-based healthcare services.

The application shall support adding, updating, searching, sorting, merging, analysing and storing patient and appointment information. Appropriate operators, decision-making constructs and looping structures are used to validate patient details, appointment availability, consultation fees and emergency priorities.

Patient and appointment details are maintained using arrays and strings, including patient ID, patient name, age, gender, contact details, department, doctor name, appointment date, consultation type, medical condition and priority level. Searching and sorting are supported using patient ID, name and age, with appointment and department information included in the maintained records.

The system supports merging patient records from different branches while identifying duplicate patient IDs. User-defined functions are used for major operations. Pointers and recursion are demonstrated for selected operations, and file handling is used to permanently store and retrieve records and generate reports.

## 2. OBJECTIVE

- Develop a menu-driven C application for hospital patient and appointment management.
- Maintain patient, doctor and appointment information using arrays, strings and structures.
- Support searching, sorting, updating and merging of records with duplicate-ID prevention.
- Validate appointment availability, consultation fees and emergency priority.
- Demonstrate functions, pointers and recursion in meaningful operations.
- Use file handling for permanent storage and retrieval of records.

• Generate a consolidated report containing key administrative statistics.

## 3. REQUIREMENTS AND ENVIRONMENT USED

| Category | Requirement / Environment |
|---|---|
| Language | C (C11) |
| Compiler | GCC |
| Operating System | Windows / Linux compatible |
| Interface | Menu-driven command-line application |
| Storage | patients.dat, doctors.dat, appointments.dat |
| Concepts | Operators, decisions, loops, arrays, strings, structures, searching, sorting, merging, functions, pointers, recursion and file handling |

## 4. DESIGN / PROPOSED SOLUTION

### 4.1 Module Design

| Module | Function |
|---|---|
| Patient Management | Add, search and update patient details. |
| Doctor Management | Add doctors and maintain department and fee information. |
| Appointment Management | Schedule, validate, classify and cancel appointments. |
| Search / Sort | Search by ID/name and sort patient records by age/name. |
| Record Merge | Merge branch patient records and skip duplicate patient IDs. |
| Report Analysis | Calculate totals, priority counts, cancellations and department-wise appointments. |
| File Management | Persist and retrieve patient, doctor and appointment records. |

### 4.2 Data Design

Three structures are used: Patient, Doctor and Appointment. Patient contains identification, demographic, contact, department, condition and follow-up fields. Doctor contains identification, name, department and consultation fee. Appointment connects a patient and doctor with date, consultation type, priority, status and fee.

## 5. ALGORITHM / PSEUDOCODE / FLOWCHART

### 5.1 Main Algorithm

1. Start the program.

2. Load previously stored records from the data files.

3. Display the menu and read a validated choice.

4. Execute the selected patient, doctor, appointment, search, sort, merge or report operation.

5. During appointment scheduling, verify patient and doctor IDs and check doctor availability.

6. Classify the appointment as Normal, Priority or Emergency.

7. Save modified records to the appropriate files.

8. Display the result and return to the menu.

9. Repeat until Exit is selected, then save data and stop.

### 5.2 Priority Classification

- If Emergency is selected, classify the appointment as Emergency.
- Otherwise, if age is 65 or above, or the condition contains 'urgent' or 'critical', classify it as Priority.
- Otherwise classify it as Normal.
- Apply the corresponding fee adjustment and store the appointment.

### 5.3 Flowchart

**START → LOAD RECORDS → DISPLAY MENU → READ / VALIDATE CHOICE → SELECT MODULE → PROCESS → SAVE RECORDS → DISPLAY RESULT → MENU → EXIT → SAVE → STOP**

# 6. IMPLEMENTATION / SOURCE CODE

## 6.1 Programming Concepts Demonstrated

- Structures and arrays store multiple patient, doctor and appointment records.
- Strings store names, departments, conditions, dates and statuses.
- Functions divide the program into reusable modules.
- Pointers update a selected Patient structure directly.
- Recursion counts emergency appointments for the consolidated report.
- qsort() supports sorting patient records by age or name.
- File handling stores and retrieves records permanently.
- Loops and decision-making constructs validate inputs and process records.

## 6.2 Source Code

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>
#define MAX_PATIENTS 200
#define MAX_DOCTORS 50
#define MAX_APPOINTMENTS 300
#define MAX_DEPARTMENTS 20
#define NAME_LEN 60
#define DATA_FILE "patients.dat"
#define DOCTOR_FILE "doctors.dat"
#define APPOINTMENT_FILE "appointments.dat"

typedef struct { int id; char name[NAME_LEN]; int age; char gender[12]; char contact[20]; char
department[40]; char medicalCondition[80]; int followUp; } Patient;
typedef struct { int id; char name[NAME_LEN]; char department[40]; float consultationFee; } Doctor;
typedef struct { int appointmentId, patientId, doctorId; char date[15]; char consultationType[20];
int priority; char status[15]; float fee; } Appointment;

Patient patients[MAX_PATIENTS]; Doctor doctors[MAX_DOCTORS]; Appointment
appointments[MAX_APPOINTMENTS];
int patientCount, doctorCount, appointmentCount;

void readLine(const char *p,char *b,size_t n){ printf("%s",p);
if(!fgets(b,(int)n,stdin)){b[0]=0;return;} b[strcspn(b,"\n")]=0; }
int readInt(const char *p,int lo,int hi){char s[64],x;int v;for(;;){readLine(p,s,sizeof
s);if(sscanf(s,"%d %c",&v,&x)==1&&v>=lo&&v<=hi)return v;printf("Invalid input. Enter %d-
%d.\n",lo,hi);}}
float readFloat(const char *p){char s[64],x;float v;for(;;){readLine(p,s,sizeof s);if(sscanf(s,"%f
%c",&v,&x)==1&&v>=0)return v;puts("Invalid fee.");}}
void pauseScreen(void){char s[4];readLine("Press ENTER to continue...",s,sizeof s);}
int containsIC(const char *a,const char *b){char x[128],y[128];size_t i;snprintf(x,sizeof
x,"%s",a);snprintf(y,sizeof y,"%s",b);for(i=0;x[i];i++)x[i]=(char)tolower((unsigned
char)x[i]);for(i=0;y[i];i++)y[i]=(char)tolower((unsigned char)y[i]);return strstr(x,y)!=NULL;}
const char *priorityName(int p){return p==3?"Emergency":p==2?"Priority":"Normal";}
int findPatient(int id){for(int i=0;i<patientCount;i++)if(patients[i].id==id)return i;return -1;}
int findDoctor(int id){for(int i=0;i<doctorCount;i++)if(doctors[i].id==id)return i;return -1;}
int findAppointment(int id){for(int
i=0;i<appointmentCount;i++)if(appointments[i].appointmentId==id)return i;return -1;}
```

```
void saveData(void){FILE *f=fopen(DATA_FILE,"wb");if(f){fwrite(&patientCount,sizeof
patientCount,1,f);fwrite(patients,sizeof(Patient),patientCount,f);fclose(f);}f=fopen(DOCTOR_FILE,"wb
");if(f){fwrite(&doctorCount,sizeof
doctorCount,1,f);fwrite(doctors,sizeof(Doctor),doctorCount,f);fclose(f);}f=fopen(APPOINTMENT_FILE,"w
b");if(f){fwrite(&appointmentCount,sizeof
appointmentCount,1,f);fwrite(appointments,sizeof(Appointment),appointmentCount,f);fclose(f);}}
void loadData(void){FILE *f=fopen(DATA_FILE,"rb");if(f){fread(&patientCount,sizeof
patientCount,1,f);if(patientCount<0||patientCount>MAX_PATIENTS)patientCount=0;fread(patients,sizeof(
Patient),patientCount,f);fclose(f);}f=fopen(DOCTOR_FILE,"rb");if(f){fread(&doctorCount,sizeof
doctorCount,1,f);if(doctorCount<0||doctorCount>MAX_DOCTORS)doctorCount=0;fread(doctors,sizeof(Doctor
),doctorCount,f);fclose(f);}f=fopen(APPOINTMENT_FILE,"rb");if(f){fread(&appointmentCount,sizeof
appointmentCount,1,f);if(appointmentCount<0||appointmentCount>MAX_APPOINTMENTS)appointmentCount=0;fr
ead(appointments,sizeof(Appointment),appointmentCount,f);fclose(f);}}

void addPatient(void){if(patientCount>=MAX_PATIENTS){puts("Patient storage full.");return;}Patient
*p=&patients[patientCount];p->id=readInt("Patient ID: ",1,999999);if(findPatient(p-
>id)>=0){puts("Duplicate Patient ID. Record rejected.");return;}readLine("Name: ",p->name,sizeof p-
>name);p->age=readInt("Age: ",1,120);readLine("Gender: ",p->gender,sizeof p-
>gender);readLine("Contact: ",p->contact,sizeof p->contact);readLine("Department: ",p-
>department,sizeof p->department);readLine("Medical condition: ",p->medicalCondition,sizeof p-
>medicalCondition);p->followUp=readInt("Follow-up required? (1/0):
",0,1);patientCount++;saveData();puts("Patient added successfully.");}
void searchPatient(void){int c=readInt("Search 1-ID 2-Name: ",1,2);if(c==1){int id=readInt("Patient
ID: ",1,999999),i=findPatient(id);if(i<0){puts("Patient not found.");return;}Patient
*p=&patients[i];printf("ID: %d\nName: %s\nAge: %d\nGender: %s\nContact: %s\nDepartment:
%s\nCondition: %s\nFollow-up: %s\n",p->id,p->name,p->age,p->gender,p->contact,p->department,p-
>medicalCondition,p->followUp?"Yes":"No");}else{char k[NAME_LEN];int found=0;readLine("Name or part
of name: ",k,sizeof k);for(int i=0;i<patientCount;i++)if(containsIC(patients[i].name,k)){printf("%d
| %-20s | %d |
%s\n",patients[i].id,patients[i].name,patients[i].age,patients[i].department);found=1;}if(!found)put
s("No matching patient.");}}
void updatePatient(Patient *p){int c=readInt("1-Contact 2-Condition 3-Follow-up:
",1,3);if(c==1)readLine("New contact: ",p->contact,sizeof p->contact);else if(c==2)readLine("New
condition: ",p->medicalCondition,sizeof p->medicalCondition);else p->followUp=readInt("Follow-up
(1/0): ",0,1);}
void updatePatientRecord(void){int id=readInt("Patient ID:
",1,999999),i=findPatient(id);if(i<0){puts("Patient not
found.");return;}updatePatient(&patients[i]);saveData();puts("Patient updated.");}

void addDoctor(void){if(doctorCount>=MAX_DOCTORS){puts("Doctor storage full.");return;}Doctor
*d=&doctors[doctorCount];d->id=readInt("Doctor ID: ",1,99999);if(findDoctor(d-
>id)>=0){puts("Duplicate Doctor ID.");return;}readLine("Doctor name: ",d->name,sizeof d-
>name);readLine("Department: ",d->department,sizeof d->department);d-
>consultationFee=readFloat("Consultation fee: ");doctorCount++;saveData();puts("Doctor added
successfully.");}
void listDoctors(void){puts("\nID     Doctor                 Department             Fee");puts("----
-----------------------------------------------------");for(int i=0;i<doctorCount;i++)printf("%-6d
%-22s %-21s
%.2f\n",doctors[i].id,doctors[i].name,doctors[i].department,doctors[i].consultationFee);}
int doctorBusy(int did,const char *date){for(int
i=0;i<appointmentCount;i++)if(appointments[i].doctorId==did&&!strcmp(appointments[i].date,date)&&str
cmp(appointments[i].status,"Cancelled"))return 1;return 0;}
int classify(int age,const char *cond,int emergency){if(emergency)return
3;if(age>=65||containsIC(cond,"critical")||containsIC(cond,"urgent"))return 2;return 1;}
void scheduleAppointment(void){if(appointmentCount>=MAX_APPOINTMENTS){puts("Appointment storage
full.");return;}int pid=readInt("Patient ID: ",1,999999),pi=findPatient(pid);if(pi<0){puts("Patient
not found.");return;}int did=readInt("Doctor ID: ",1,99999),di=findDoctor(did);if(di<0){puts("Doctor
not found.");return;}Appointment *a=&appointments[appointmentCount];a-
>appointmentId=1001+appointmentCount;a->patientId=pid;a->doctorId=did;readLine("Date (DD-MM-YYYY):
",a->date,sizeof a->date);if(doctorBusy(did,a->date)){puts("Doctor is unavailable on that
date.");return;}readLine("Consultation type: ",a->consultationType,sizeof a->consultationType);int
e=readInt("Emergency? (1/0): ",0,1);a-
>priority=classify(patients[pi].age,patients[pi].medicalCondition,e);a-
>fee=doctors[di].consultationFee+(a->priority==3?200.0f:a->priority==2?100.0f:0.0f);strcpy(a-
>status,"Scheduled");appointmentCount++;saveData();printf("Appointment %d scheduled. Priority: %s,
Fee: %.2f\n",a->appointmentId,priorityName(a->priority),a->fee);}
void cancelAppointment(void){int id=readInt("Appointment ID:
",1,999999),i=findAppointment(id);if(i<0){puts("Appointment not
found.");return;}if(!strcmp(appointments[i].status,"Cancelled")){puts("Already
cancelled.");return;}strcpy(appointments[i].status,"Cancelled");saveData();puts("Appointment
cancelled successfully.");}

int cmpAge(const void *a,const void *b){return ((const Patient*)a)->age-((const Patient*)b)->age;}
int cmpName(const void *a,const void *b){return strcmp(((const Patient*)a)->name,((const
Patient*)b)->name);}
```

```
void sortPatients(void){if(patientCount<2){puts("Not enough records.");return;}int c=readInt("Sort
1-Age 2-Name:
",1,2);qsort(patients,patientCount,sizeof(Patient),c==1?cmpAge:cmpName);saveData();puts("Records
sorted.");}
void listAppointments(void){puts("\nID    Patient              Doctor               Date
Priority     Fee       Status");puts("-----------------------------------------------------------------
------------------------");for(int i=0;i<appointmentCount;i++){int
p=findPatient(appointments[i].patientId),d=findDoctor(appointments[i].doctorId);printf("%-5d %-20s
%-20s %-12s %-12s %-9.2f
%s\n",appointments[i].appointmentId,p>=0?patients[p].name:"Unknown",d>=0?doctors[d].name:"Unknown",a
ppointments[i].date,priorityName(appointments[i].priority),appointments[i].fee,appointments[i].statu
s);}}

void mergeBranchFile(void){char file[120];readLine("Branch patient file: ",file,sizeof file);FILE
*f=fopen(file,"rb");if(!f){puts("Branch file not found.");return;}Patient p;int
add=0,dup=0;while(fread(&p,sizeof p,1,f)==1){if(findPatient(p.id)>=0)dup++;else
if(patientCount<MAX_PATIENTS){patients[patientCount++]=p;add++;}}fclose(f);saveData();printf("Merge
complete. Imported: %d, duplicates skipped: %d\n",add,dup);}
int emergencyRecursive(int i){if(i>=appointmentCount)return 0;return
(appointments[i].priority==3)+emergencyRecursive(i+1);}
void report(void){int normal=0,priority=0,cancelled=0,follow=0;char dept[MAX_DEPARTMENTS][40];int
dc[MAX_DEPARTMENTS]={0},dn=0;for(int i=0;i<patientCount;i++)if(patients[i].followUp)follow++;for(int
i=0;i<appointmentCount;i++){if(appointments[i].priority==1)normal++;else
if(appointments[i].priority==2)priority++;if(!strcmp(appointments[i].status,"Cancelled"))cancelled++
;int p=findPatient(appointments[i].patientId);if(p>=0){int j=-1;for(int
k=0;k<dn;k++)if(!strcmp(dept[k],patients[p].department))j=k;if(j<0&&dn<MAX_DEPARTMENTS){strcpy(dept[
dn],patients[p].department);dc[dn++]=1;}else if(j>=0)dc[j]++;}}puts("\n========== CONSOLIDATED
REPORT ==========");printf("Total patients: %d\nTotal doctors: %d\nTotal appointments: %d\nNormal:
%d\nPriority: %d\nEmergency: %d\nCancelled: %d\nFollow-up patients:
%d\n",patientCount,doctorCount,appointmentCount,normal,priority,emergencyRecursive(0),cancelled,foll
ow);puts("Department-wise appointments:");for(int i=0;i<dn;i++)printf("  %-25s
%d\n",dept[i],dc[i]);puts("========================================");}
void demo(void){if(patientCount||doctorCount||appointmentCount){puts("Demo data not loaded because
records already exist.");return;}patients[0]=(Patient){101,"Arun
Kumar",45,"Male","9876543210","Cardiology","Chest pain",1};patients[1]=(Patient){102,"Meena
Devi",68,"Female","9876501234","Neurology","Urgent
headache",1};patients[2]=(Patient){103,"Rahul",30,"Male","9000012345","Orthopaedics","Fracture",0};p
atientCount=3;doctors[0]=(Doctor){201,"Dr. Priya","Cardiology",800};doctors[1]=(Doctor){202,"Dr.
Kumar","Neurology",900};doctors[2]=(Doctor){203,"Dr.
Anitha","Orthopaedics",700};doctorCount=3;appointments[0]=(Appointment){1001,101,201,"02-09-
2026","In-person",1,"Scheduled",800};appointments[1]=(Appointment){1002,102,202,"02-09-2026","In-
person",2,"Scheduled",1000};appointments[2]=(Appointment){1003,103,203,"03-09-
2026","Online",1,"Cancelled",700};appointmentCount=3;saveData();puts("Demo data loaded
successfully.");}
void menu(void){puts("\n============================================");puts(" SMART HEALTHCARE
PATIENT MANAGEMENT SYSTEM");puts("============================================");puts("1 Add
Patient\n2 Search Patient\n3 Update Patient\n4 Add Doctor\n5 List Doctors\n6 Schedule Appointment\n7
Cancel Appointment\n8 List Appointments\n9 Sort Patients\n10 Merge Branch Patient File\n11 Generate
Consolidated Report\n12 Load Demo Data\n0 Exit");}
int main(void){loadData();int c;do{menu();c=readInt("Enter choice: ",0,12);switch(c){case
1:addPatient();break;case 2:searchPatient();break;case 3:updatePatientRecord();break;case
4:addDoctor();break;case 5:listDoctors();break;case 6:scheduleAppointment();break;case
7:cancelAppointment();break;case 8:listAppointments();break;case 9:sortPatients();break;case
10:mergeBranchFile();break;case 11:report();break;case 12:demo();break;case 0:saveData();puts("Data
saved. Program closed.");break;}if(c)pauseScreen();}while(c);return 0;}
```

## 7. TEST CASES AND EXPECTED / ACTUAL RESULTS

| ID | Test Case | Input / Action | Expected / Actual Result | Status |
|---|---|---|---|---|
| TC01 | Load demonstration data | Menu 12 | Demo records loaded successfully | Pass |
| TC02 | Search patient | Search ID 101 | Arun Kumar record displayed | Pass |
| TC03 | Duplicate ID | Add existing | Record rejected | Pass |

| | | | | |
|---|---|---|---|---|
| | | Patient ID 101 | | |
| TC04 | Emergency classification | Emergency = 1 | Appointment classified as Emergency | Pass |
| TC05 | Doctor availability | Same doctor and same date | Second active appointment rejected | Pass |
| TC06 | Cancellation | Cancel Appointment 1003 | Status becomes Cancelled | Pass |
| TC07 | Sorting | Sort by age or name | Patient order changes correctly | Pass |
| TC08 | Consolidated report | Menu 11 | Totals and department counts displayed | Pass |
| TC09 | Invalid menu choice | Value outside 0–12 | Validation message displayed | Pass |

## 8. EXECUTION SCREENSHOTS / OUTPUT

The program was compiled with GCC using C11 and executed with demonstration data. The screenshot below shows the menu operation, appointment listing and consolidated report output.

**Smart Healthcare Patient Appointment System – Execution Output**

```
================================================
 SMART HEALTHCARE PATIENT MANAGEMENT SYSTEM
================================================
1 Add Patient
2 Search Patient
3 Update Patient
4 Add Doctor
5 List Doctors
6 Schedule Appointment
7 Cancel Appointment
8 List Appointments
9 Sort Patients
10 Merge Branch Patient File
11 Generate Consolidated Report
12 Load Demo Data
0 Exit
Enter choice: Demo data loaded successfully.
Press ENTER to continue...
================================================
 SMART HEALTHCARE PATIENT MANAGEMENT SYSTEM
================================================
1 Add Patient
2 Search Patient

...

Cancelled: 1
Follow-up patients: 2
Department-wise appointments:
  Cardiology                1
  Neurology                 1
  Orthopaedics              1
========================================
Press ENTER to continue...
================================================
 SMART HEALTHCARE PATIENT MANAGEMENT SYSTEM
================================================
1 Add Patient
2 Search Patient
3 Update Patient
4 Add Doctor
5 List Doctors
6 Schedule Appointment
7 Cancel Appointment
8 List Appointments
9 Sort Patients
10 Merge Branch Patient File
11 Generate Consolidated Report
12 Load Demo Data
0 Exit
Enter choice: Data saved. Program closed.
```

## 9. ANALYSIS AND DISCUSSION

The system satisfies the major functional requirements in the supplied assignment. Structures organize related fields, arrays provide in-memory storage, and functions separate the major operations. Input validation reduces invalid records and duplicate patient IDs are rejected.

Appointment scheduling verifies that the patient and doctor exist and prevents an active duplicate appointment for the same doctor and date. Priority classification uses decision-

making based on emergency selection, age and medical-condition keywords. Searching, sorting and merging demonstrate data-processing operations required by the assignment.

Pointers are demonstrated during patient updates, while recursion is used to analyse emergency appointments. Binary file handling provides persistence between executions. The implementation is an academic prototype; a production hospital system would require database management, stronger validation, authentication, privacy protection and clinically approved workflows.

## 10. CONCLUSION

The Smart Healthcare Patient Appointment and Medical Record Management System demonstrates a practical menu-driven application in C. It integrates patient, doctor and appointment management with validation, searching, sorting, merging, priority classification, pointers, recursion and file handling. The prototype provides a clear implementation of the programming concepts specified in the assignment.

## 11. INDIVIDUAL CONTRIBUTION OF GROUP MEMBERS

| **Member** | **Contribution** |
|---|---|
| Vikash J | System design, C implementation, integration and testing. |
| Jafar Husen | Patient, doctor and appointment module support and test-case preparation. |
| Tharun Kumar | Output verification, analysis and assignment documentation support. |

## 12. REFERENCES


- C Programming classroom notes and laboratory materials.
- C standard library documentation for stdio.h, stdlib.h, string.h and ctype.h.
- Reference material on C structures, pointers, recursion, sorting and file handling.

## 13. ONE-PAGE WRITE-UP

The Smart Healthcare Patient Appointment and Medical Record Management System is a menu-driven C application designed to help hospital administrators maintain patient, doctor and appointment information. The system provides a structured way to add, search, update, sort, merge and analyse records.

The application uses structures for Patient, Doctor and Appointment records. Arrays store multiple records, while strings represent names, departments, medical conditions, dates and status values. User-defined functions divide the application into clear modules. Validation is applied to numeric input, duplicate patient IDs and appointment availability.

Appointments are classified as Normal, Priority or Emergency. An explicitly marked emergency case receives the highest priority. Otherwise, elderly patients and patients with urgent or critical conditions receive Priority classification. Consultation fees are adjusted according to the classification in the prototype.

The required C programming concepts are demonstrated directly in the implementation. Pointers are used to update patient records, recursion is used to count emergency appointments, qsort() is used for sorting, and file handling stores patient, doctor and appointment records permanently. A merge operation imports branch patient records while preventing duplicate patient IDs.

The consolidated report displays total patients, doctors and appointments, priority counts, emergency cases, cancelled appointments, follow-up patients and department-wise appointment counts. The completed system therefore combines the specified programming concepts into one practical healthcare-management prototype. Future versions could add a database, authentication, stronger data validation and secure hospital information management.